

Relatório

Alunos: Fernando Infantini, Giovana Piazza, Ronaldy Gelos
Instituição: Universidade Federal do Pampa (Unipampa) - Campus Bagé

01 de Julho de 2026

Este documento apresenta uma visão geral do sistema desenvolvido. O sistema consiste em uma aplicação que simula a alocação de memória em uma heap controlada, implementada como um vetor de inteiros. O software é capaz de gerar requisições automáticas, realizar alocação conforme algoritmos de política contígua e liberação de espaço de forma aleatória. O gerenciamento de memória é um dos principais desafios em sistemas operacionais. A fragmentação e a disputa por recursos em ambientes concorrentes tornam o desenvolvimento de alocadores eficientes uma tarefa complexa que exige análise empírica de desempenho.

O objetivo geral deste trabalho consiste no desenvolvimento de um simulador voltado à modelagem do gerenciamento dinâmico de memória contígua. A aplicação projeta e executa requisições automatizadas de carga com tamanhos variáveis, garantindo que cada bloco ocupado possua um identificador (ID) exclusivo gravado diretamente na estrutura de dados do sistema. Esta Heap controlada é implementada de forma independente como um vetor primitivo de inteiros, desvinculando-se totalmente do comportamento nativo da Máquina Virtual Java (JVM). Para mitigar os efeitos da fragmentação externa, o software incorpora a política de alocação Best-Fit integrada a rotinas de compactação de memória. Ademais, em cenários de saturação física do vetor, o gerenciador ativa uma rotina de contingência baseada na desalocação aleatória (RANDOM) de, no mínimo, 30% da capacidade total da estrutura. Por fim, o cerne da pesquisa reside em comparar, de forma empírica, o desempenho das execuções sequenciais e paralelas, operadas por múltiplas threads concorrentes e sincronizadas por semáforos, consolidando os resultados analíticos por meio de métricas de tempo de execução, taxa de throughput e índices de fragmentação em relatórios estatísticos detalhados.

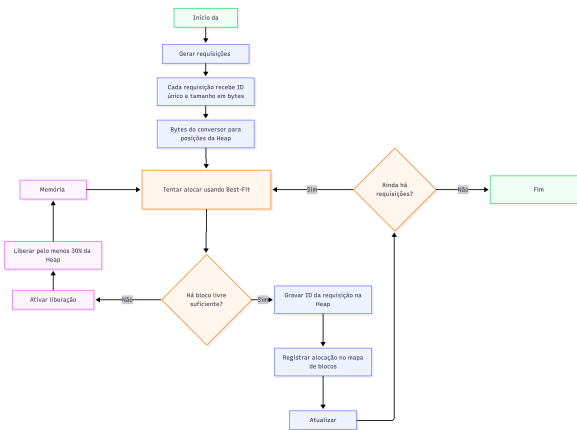


Figura 1: Objetivos do projeto

Para facilitar a compreensão do funcionamento do código, a explicação do simulador foi feito um diagrama 1 que representa o funcionamento interno do gerenciamento de memória, contemplando a geração das requisições, a conversão dos tamanhos para posições da Heap, a aplicação da política Best-Fit, a gravação dos IDs nos blocos ocupados, além das rotinas de liberação RANDOM de 30% e compactação quando não há espaço suficiente.

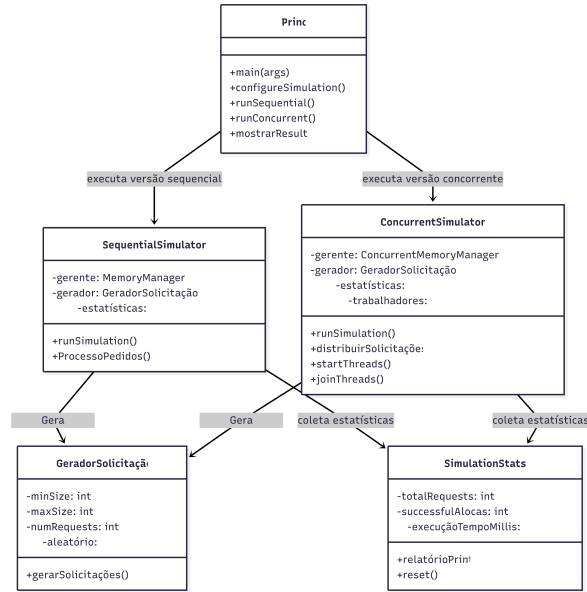


Figura 2: Diagrama de Uso

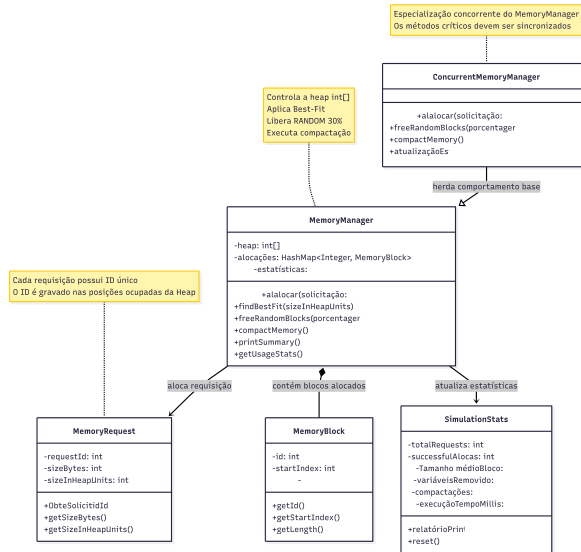


Figura 3: Diagrama de Uso

Durante o trabalho, foram elaborados alguns diagramas com o objetivo de facilitar o entendimento da estrutura e do funcionamento do simulador. Esses diagramas foram divididos em partes menores para tornar a visualização mais clara e evitar o excesso de informações em uma única representação. O primeiro diagrama 2 apresenta uma visão geral da simulação, mostrando como a classe principal inicia o sistema e separa a execução entre a versão sequencial e a versão concorrente. O segundo diagrama 3 representa o núcleo do gerenciamento de memória, destacando a Heap, as requisições, os blocos alocados, a política Best-Fit, a liberação RANDOM de 30% e a compactação. Já o terceiro diagrama 4 enfatiza a execução concorrente, demonstrando a atuação das múltiplas threads, o acesso compartilhado ao gerenciador de memória e a necessidade de sincronização nas regiões críticas. Dessa forma, os diagramas auxiliam na compreensão tanto da organização interna do código quanto dos principais conceitos de Sistemas Operacionais aplicados no projeto.

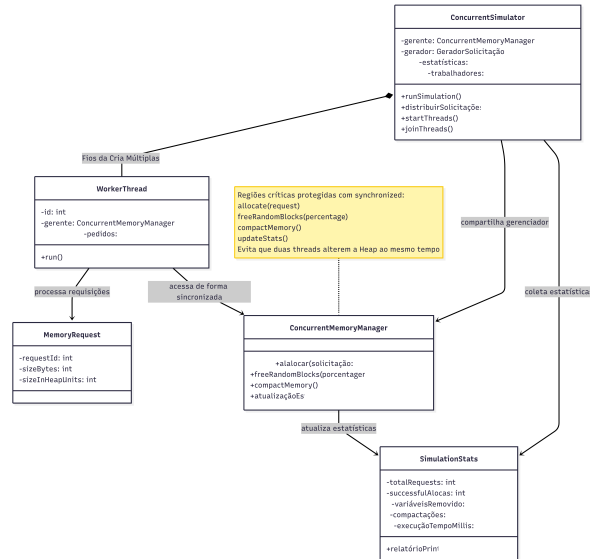


Figura 4: Diagrama de Uso

O projeto envolve uma equipe de stakeholders composta pelos desenvolvedores Fernando Infantini, Giovana Piazza e Ronaldy Gelos, que atuam de forma integrada no planejamento, desenvolvimento e validação do simulador. O sistema foi projetado para atender principalmente a dois perfis de usuários: o configurador e o avaliador. O configurador é responsável por definir os parâmetros operacionais da simulação, como o tamanho total da Heap em KB, os limites mínimo e máximo para o tamanho das alocações e a quantidade de threads simultâneas utilizadas nos testes de carga. Já o avaliador tem como principal função analisar os relatórios estatísticos e os comparativos de desempenho gerados automaticamente ao final das simulações, permitindo observar o comportamento do sistema nas versões sequencial e concorrente.

As funcionalidades principais do sistema contemplam a configuração dinâmica da Heap e de seus parâmetros de carga (RF01 e RF03), a geração de identificadores únicos escritos em cada posição ocupada para garantir a integridade visual da ocupação (RF04) e a presença de algoritmos específicos para a política de alocação Best-Fit (RF05), liberação emergencial RANDOM acionada por um gatilho obrigatório de 30% da capacidade total da memória (RF06) e compactação para eliminação de fragmentação externa (RF07). Ao final, o sistema opera de modo sequencial para fins de baseline (RF08) ou concorrente (RF09), emitindo um relatório estatístico detalhado que apresenta o tempo total de execução, a média dos tamanhos das requisições atendidas e a contagem de chamadas de rotinas de faxina e compactação (RF10).

Os requisitos não funcionais, destaca-se o uso obrigatório de concorrência por meio de múltiplas threads para simular uma carga de trabalho real (RF09), bem como a necessidade imperativa de sincronização de dados para garantir a exclusão mútua nas regiões críticas da Heap. A interface deve fornecer uma apresentação clara dos resultados obtidos via console ou por meio de uma estrutura simples utilizando ferramentas como Streamlit ou JavaFX. Adicionalmente, o processo de gerenciamento do desenvolvimento é estritamente orientado pelo framework Scrum, exigindo artefatos, cerimônias e papéis definidos ao longo de suas etapas.

O projeto está sujeito a restrições técnicas e metodológicas rígidas: a implementação deve ocorrer obrigatoriamente na linguagem Java. A Heap deve ser modelada de forma independente como um vetor primitivo de inteiros (`int[]`), reservando exatamente 4 bytes por posição para se desvincular completamente da Heap padrão da JVM. Para o controle de concorrência, exige-se o uso exclusivo de mecanismos de sincronização de baixo nível, tais como Semáforos ou Monitores, sendo proibido o uso de bibliotecas concorrentes de alto nível. Na esfera de gestão e documentação, cada membro da equipe deve atuar como Scrum Master em ao menos uma etapa, promovendo a rotação obrigatória entre os integrantes; o código fonte deve estar plenamente comentado; o site do projeto precisa ser mantido

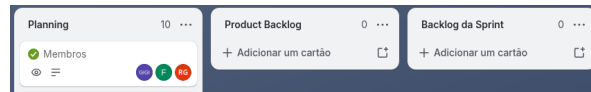


Figura 5: Colunas do Trello

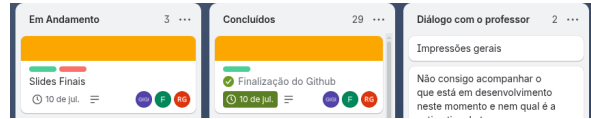


Figura 6: Colunas do Trello

atualizado para acompanhamento docente; e todos os slides e entregáveis dos seminários acadêmicos devem ser submetidos obrigatoriamente no formato de arquivo .ppt.

O processo de desenvolvimento do software foi orientado pelo framework Scrum, adaptado para um ciclo composto por cinco Sprints com durações variáveis, definidas de acordo com a complexidade das entregas previstas em cada etapa. O cronograma de execução foi organizado da seguinte forma: a Sprint 1 teve duração de duas semanas, a Sprint 2 também foi planejada para duas semanas, a Sprint 3 teve duração de três semanas, a Sprint 4 foi estruturada em quatro semanas e a Sprint 5 foi concluída em três semanas. Essa distribuição buscou garantir maior flexibilidade ao time, permitindo que as etapas mais complexas recebessem mais tempo de desenvolvimento, sem comprometer a agilidade operacional e o cumprimento das metas estabelecidas para o projeto.

Os papéis internos da equipe foram definidos com o objetivo de assegurar a colaboração entre os integrantes e a organização contínua do fluxo de trabalho. A função de Scrum Master foi atribuída de forma rotativa, permitindo que diferentes membros assumissem a liderança ao longo das Sprints. Esse papel envolveu a organização das atividades, o acompanhamento do andamento do projeto, a marcação de reuniões e a verificação de que as restrições técnicas estabelecidas estavam sendo seguidas. O papel de Product Owner foi desempenhado de maneira compartilhada por todos os integrantes do grupo, que atuaram em conjunto na interpretação e validação das demandas propostas pelos professores das disciplinas. Já o Time de Desenvolvimento foi composto por todos os membros da equipe, incluindo o Scrum Master ativo em cada Sprint, sendo responsável pela codificação do simulador, pela implementação das funcionalidades previstas e pela elaboração dos artefatos de engenharia necessários para documentar o projeto.

Nas imagens 5 e 6 estão representadas as colunas do Trello utilizadas para organizar visualmente as tarefas do projeto. O quadro foi estruturado seguindo o modelo Kanban, com seis colunas principais, cada uma com uma função específica no acompanhamento do desenvolvimento. A coluna Planning é destinada à organização das sprints e ao agendamento de reuniões. A coluna Product Backlog centraliza todas as tarefas macro ainda pendentes do projeto, enquanto a coluna Sprint Backlog reúne os itens selecionados para serem executados no ciclo atual. Já a coluna Em Andamento concentra as funcionalidades e documentos que estão sendo desenvolvidos no momento. Após a finalização, teste e documentação das tarefas, elas são movidas para a coluna Concluído. Além disso, foi criada a coluna Diálogo com o Professor, utilizada para registrar comentários, feedbacks e apontamentos feitos pelo docente ao longo do projeto.

O ecossistema de ferramentas adotado pela equipe também inclui o Discord, utilizado para reuniões, o WhatsApp, utilizado para comunicações rápidas entre os integrantes, e o GitHub, definido como repositório oficial do código-fonte (<https://github.com/giovanapiazza/Sistemas-Operacionais-e-Engenharia-de-Software>).

No diagrama de Gantt, representado pela Figura 7, é demonstrado como as Sprints foram divididas ao longo do desenvolvimento do projeto, evidenciando a duração de cada etapa, a sequência das atividades e os membros responsáveis pela liderança em cada período. O processo de desenvolvimento do software foi orientado pelo framework Scrum, adaptado para um ciclo composto por cinco Sprints com durações variáveis, definidas de acordo com a complexidade das entregas principais de cada fase. A Sprint 1 teve duração de duas semanas, assim como a Sprint 2; a Sprint 3 foi planejada para três

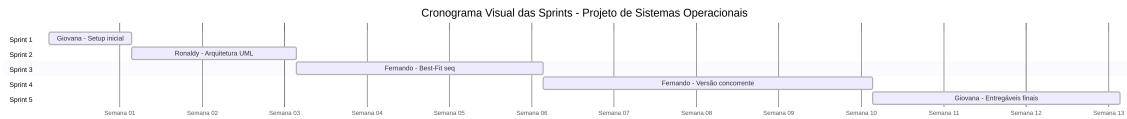


Figura 7: Cronograma Visual

semanas; a Sprint 4, por envolver a transição crítica para a versão concorrente do simulador, teve duração de quatro semanas; e a Sprint 5 foi concluída em três semanas, concentrando-se nos testes, métricas e entregáveis finais. Essa variação temporal teve como objetivo garantir maior flexibilidade ao time, mantendo a agilidade operacional e o cumprimento das metas estabelecidas.

O histórico de papéis registra a trajetória de liderança e a dinâmica de organização da equipe ao longo das iterações planejadas. Na Sprint 1, Giovana assumiu a função de Scrum Master, ficando responsável pelo setup inicial do ambiente, pela criação do Documento de Visão e pela organização estrutural do quadro no Trello. Na Sprint 2, Ronaldy assumiu a liderança do grupo durante a fase de modelagem da arquitetura e elaboração dos diagramas UML essenciais, como os diagramas de Classes, Casos de Uso e Sequência. Na Sprint 3, Fernando coordenou a equipe durante a implementação da lógica sequencial do algoritmo Best-Fit, das rotinas de faxina emergencial RANDOM de 30% e da compactação de memória. Na Sprint 4, Fernando permaneceu na liderança, conduzindo a transição do simulador para o ambiente concorrente, com múltiplas threads e mecanismos de sincronização por semáforos. Por fim, na Sprint 5, Giovana retornou à função de Scrum Master, direcionando os esforços finais do grupo para a consolidação dos testes de carga paralelizada, a coleta de métricas para os relatórios estatísticos de desempenho e a preparação final dos entregáveis do projeto.

Referências

- [1] SILBERSCHATZ, A.; GALVIN, P. B.; GAGNE, G. **Operating System Concepts**. 10. ed. Hoboken: Wiley, 2018.
- [2] TANENBAUM, A. S.; BOS, H. **Modern Operating Systems**. 4. ed. Boston: Pearson, 2015.
- [3] GOETZ, B.; PEIERLS, T.; BLOCH, J.; BOWBEER, J.; HOLMES, D.; LEA, D. **Java Concurrency in Practice**. Boston: Addison-Wesley Professional, 2006.
- [4] SOMMERVILLE, I. **Engineering Software Products: An Introduction to Modern Software Engineering**. Boston: Pearson, 2019.